Cyber Deception AI — Complete Viva Guide

Project: AI-Based Cyber Deception System for Intelligent Honeypot Placement and Attacker Path Prediction
Student: Chidwilas Routhu · 24BYB0092 · VIT Vellore
Repository: `D:\cyber-deception-ai` (GitHub: `sid07-pro/cyber-deception-ai`)

> **Ground rule for the whole viva:** every number in this guide comes from running the project's own code on its built-in network on 2026-10-21. If a question asks for something the project does not do, say so plainly. Honesty about scope is a strength, not a weakness.

Part A — Project fundamentals

A1. Motivation

Prevention controls are essential: firewalls, patching and access control. Once an attacker has a foothold, though, defenders often have little visibility into lateral movement. Honeypots give high-signal alerts, because nobody legitimate should ever touch them. But they only help if they sit where the attacker will actually go. With a limited budget, **placement** is the real decision.

A2. Problem statement

> Given a network, an attacker entry point, a critical asset and a honeypot budget *k*: which route is the attacker most likely to take, and which *k* hosts should become honeypots so that route is intercepted before the asset is reached?

A3. Objectives (all implemented and tested)

1. **Model** the network as a weighted graph with roles.
2. **Predict** the attacker route with BFS, DFS, UCS and A*, plus the 3 cheapest alternatives.
3. **Place** honeypots with a transparent greedy score, and compare it with Hill Climbing and a Genetic Algorithm.
4. **Explain** every decision in plain language, together with its scores.
5. **Evaluate** with repeated simulated runs and automated tests.

A4. Scope

- **In scope:** a simulated network, defensive decision support, explainable algorithms, a full-stack application.
- **Out of scope:** real networks, scanning, real honeypots, telemetry, exploitation, machine learning.

A5. Cyber deception

Deliberately misleading attackers with false assets, such as hosts, files, credentials or services. The aim is to detect them, delay them, or learn their behaviour.

A6. Honeypots and honeynets

- **Honeypot:** a decoy system with no production purpose. Any interaction with it is suspicious.
- **Low-interaction** honeypots emulate services.
- **High-interaction** honeypots are real systems that are heavily monitored.
- **Honeynet:** a network of honeypots that looks like a real environment.
- **Decoy asset:** any planted fake artefact.

In this project a honeypot is simply a *label on a graph node*. The simulation checks whether the attacker's path contains that node.

A7. Threat model (as modelled)

The attacker:

- starts at `Internet`;
- wants to reach `Database`;
- knows the whole graph;
- minimises path cost with the chosen search algorithm;
- does not detect decoys.

These are simplifying assumptions. Real attackers explore, make mistakes, and can fingerprint honeypots.

A8. Attack surface

The set of points where an attacker can try to enter or move. Here, `Internet` is the only entry point, and its single link to `Web Server` is the whole external attack surface.

A9. Attack path

An ordered sequence of hosts from the entry point to the target, where each consecutive pair is linked. The **cost of a path** is the sum of its edge weights.

Part B — Graph theory

B1. Notation

A graph is **G = (V, E)**: vertices V (hosts) and edges E (links). In this project:

- |V| = 7;
- |E| = 9.

B2. Directed vs undirected

- **Undirected** means a link can be crossed in both directions. This project uses `nx.Graph`.
- **Directed** means one-way links, such as firewall rules. That is future work. The search code would still work, because it only calls `neighbors()`.

B3. Weighted graphs

Each edge has a weight w(u, v): the simulated *difficulty* of crossing it. The weights are hand-chosen modelling values, not measurements.

B4. The built-in network (`backend/simulation/network.py`)

Edge	Weight		Edge	Weight
Internet – Web Server	1		Employee PC – Admin PC	2
Web Server – Application Server	2		Admin PC – Database	4
Web Server – Employee PC	2		File Server – Database	4
Application Server – Database	5		Employee PC – File Server	3
Application Server – File Server	3			

Roles:

- `Internet` = normal (the entry point);
- `Database` = critical (the target);
- the other 5 hosts = honeypot candidates.

B5. Adjacency list vs matrix

- **Adjacency list:** neighbours per node. Space O(V + E), and iterating over a node's neighbours is fast. NetworkX uses this structure.
- **Adjacency matrix:** a V×V table. Space O(V^2), and it answers "is there an edge?" in O(1). It is wasteful for sparse networks.

B6. Reachability

v is reachable from u if a path exists between them. Search returns an empty path when the target is unreachable. The tests cover this with a disconnected graph.

B7. Cycles

A cycle is a closed path, for example Web Server → Application Server → File Server → Employee PC → Web Server. Search must track visited nodes, or it could loop forever.

B8. Example calculations

Path	Cost	Hops
Internet → Web → App → Database	1 + 2 + 5 = **8**	3
Internet → Web → Employee → Admin → Database	1 + 2 + 2 + 4 = **9**	4
Internet → Web → App → File → Database	1 + 2 + 3 + 4 = **10**	4
Internet → Web → Employee → File → Database	1 + 2 + 3 + 4 = **10**	4

- **Degree** = number of links. Web Server, Application Server, Employee PC, File Server and Database have degree 3; Admin PC has 2; Internet has 1.

- **Chokepoint:** Internet's only neighbour is Web Server, so *every* route passes through Web Server. This one fact explains many results.

Part C — Algorithms

All four searches have the same interface: `search(graph, start, goal) → (path, cost, nodes_explored)`. They are hand-written in `backend/ai/` and are not NetworkX calls. An unknown start or goal returns `([], 0, 0)`.

C1. Breadth-First Search (`ai/bfs.py`)

Idea: explore level by level. **Data structure:** a FIFO queue (`deque`), a `visited` set, and a `parent` map.

```
frontier ← queue[start]; visited ← {start}; parent[start] ← None
while frontier not empty:
    current ← frontier.popleft(); explored += 1
    if current = goal: return reconstruct(parent, goal), cost, explored
    for each neighbour n of current not in visited:
        visited.add(n); parent[n] ← current; frontier.append(n)
return [], 0, explored
```

Dry run on the built-in network. The pop order is:

1. Internet
2. Web Server
3. Application Server
4. Employee PC
5. Database

Result: 5 nodes explored, path Internet → Web → App → Database, cost 8.

Time	O(V + E)
Space	O(V)
Complete	Yes
Optimal	Fewest **hops** only, not lowest cost

C2. Depth-First Search (`ai/dfs.py`)

Idea: go as deep as possible, then backtrack. **Data structure:** an explicit stack of `(node, path)` pairs and a `visited` set. Neighbours are pushed in reverse so the first neighbour is explored first.

Dry run: 4 nodes explored (Internet, Web Server, Application Server, Database); path cost 8.

Time	O(V + E)
Space	Up to O(V) stack entries, each holding a copied path
Complete	Yes, on finite graphs with a visited set
Optimal	No. The result depends on neighbour order.

C3. Uniform-Cost Search (`ai/ucs.py`)

Idea: always expand the cheapest path found so far, g(n). This is Dijkstra's algorithm, stopped when the goal is popped. It is an **uninformed** search: it uses no estimate of the remaining distance.

Data structure: a min-heap of `(cost, counter, node, path)`, a `best_cost` dict, and a `visited` set.

```
frontier ← heap[(0, 0, start, [start])]; best_cost[start] ← 0
while frontier:
    cost, _, current, path ← heappop(frontier)
    if current in visited: continue            # stale entry
    visited.add(current); explored += 1
    if current = goal: return path, cost, explored
    for each neighbour n:
        new ← cost + w(current, n)
        if n not in best_cost or new < best_cost[n]:
            best_cost[n] ← new; push (new, counter++, n, path + [n])
```

Full dry run (this is the slide 10 table):

Step	Pop (cost)	Push / update	Frontier after
1	Internet (0)	Web = 1	(1 Web)
2	Web (1)	App = 3, Employee = 3	(3 App) (3 Emp)
3	App (3)	Database = 8, File = 6	(3 Emp) (6 File) (8 DB)
4	Employee (3)	Admin = 5; File via Employee = 6, not better	(5 Admin) (6 File) (8 DB)
5	Admin (5)	DB via Admin = 9 > 8	(6 File) (8 DB)
6	File (6)	DB via File = 10 > 8	(8 DB)
7	Database (8)	goal popped → return	—

Result: 7 nodes explored, cost 8.

Why it is correct: with non-negative weights, the first time a node is *popped* its cost is minimal. Any cheaper route would have a smaller key and would have been popped earlier.

Time	O((V + E) log V)
Space	O(V)
Optimal	Yes, for positive weights

C4. A* Search (`ai/astar.py`)

Idea: order the frontier by **f(n) = g(n) + h(n)**, where h estimates the remaining cost.

The heuristic used: the BFS hop distance to the goal × the minimum edge weight (which is 1 here). On this graph:

Node	h
Database	0
Application Server	1
Admin PC	1
File Server	1
Web Server	2
Employee PC	2
Internet	3

Admissibility: every remaining hop costs at least the minimum edge weight, so h never overestimates the true cost. That makes A* optimal.

Result: 7 nodes explored, the same as UCS. The heuristic is weak, because the real costs (2–5) far exceed 1 per hop. A *is never worse than UCS here, but it is not better either. With h = 0, A is* UCS.

Edge case: on a graph with no edges, the minimum weight defaults to 0. h is then 0 everywhere, which is still admissible.

C5. Comparison on the built-in network

Algorithm	Path	Cost	Nodes explored
BFS	Internet → Web → App → Database	8	5
DFS	same	8	4
UCS	same	8	7
A*	same	8	7

All four agree here only by coincidence: the fewest-hop route also happens to be the cheapest. The test graph `TWO_ROUTES` separates them:

- S–X–G costs 10 in 2 hops; BFS picks it.
- S–A–B–G costs 3 in 3 hops; UCS and A* pick it.

C6. Edge cases (`tests/test_search_edge_cases.py` , 31 tests)

Case	Behaviour
Unreachable target	Empty path, cost 0, explored = size of the start's component
Cycles	Terminates, returns a simple valid path
start = goal	`([start], 0, 1)`
Unknown node	`([], 0, 0)` and no exception (hardening added during recovery)
Isolated nodes, no edges	Empty path; A* does not crash
Random graphs	UCS and A* cost equal NetworkX's `shortest_path_length`

C7. Alternative routes (Yen's algorithm)

`alternative_paths()` calls `nx.shortest_simple_paths(graph, start, target, weight="weight")` . That function implements Yen's k-shortest loopless paths. The project takes the first 3 routes (costs 8, 9, 10) and reports

which honeypots each one passes through.

C8. Adversarial search

- **Game** (`ai/deception_game.py`):
- MAX is the attacker and moves one hop per turn.
- MIN is the defender and places a honeypot (within budget) or passes.
- Terminal states: intercepted scores −50, target reached scores +50.
- Otherwise, U = 10/(1+dist) + 2·|compromised| − 4·protection − 0.5·cost.
- **Minimax** explores the full tree to depth d. Time O(b^d).
- **Alpha-Beta** keeps bounds α (best value MAX is guaranteed so far) and β (best value MIN is guaranteed so far). It prunes a branch when β ≤ α. The result is the same as Minimax, with fewer nodes visited.
- **Measured at depth 4:** Minimax visits 69 states; Alpha-Beta visits 35 and prunes 14 branches. Both choose "move Web Server" with value 2.6667.

C9. Optimisers

- **Hill Climbing** (`ai/hill_climbing.py`):
- steepest ascent;
- neighbours are placements one swap, add or drop away;
- 4 random restarts;
- seeded, so runs are reproducible.
- Result: {Application Server, Employee PC}, fitness 6.2667, 44 evaluations.
- **Genetic Algorithm** (`ai/genetic.py`):
- population 12, 15 generations;
- tournament selection, crossover, mutation rate 0.2;
- a repair step keeps exactly k genes.
- Result: {File Server, Web Server}, fitness 6.2667, 194 evaluations.

C10. Planning (`ai/planning.py`)

Representation: STRIPS-style. States are sets of literals; actions have preconditions, add-effects and delete-effects.

Action	Cost
place_honeypot	2
block_route	3
increase_protection	2
monitor_node	1
isolate_node	2
move_attacker	the edge weight

Searches:

- **Forward search** runs BFS from the initial state; it explores 48 states.
- **Backward regression** runs BFS from the goal; it explores 5 states.

Result: both find the same 3-action plan, total cost 7:

1. place_honeypot(Application Server)
2. place_honeypot(File Server)
3. block_route(Admin PC, Database)

Because both searches are BFS over states, the plan has the fewest *actions* within the depth limit. It is not guaranteed to be the cheapest plan.

Part D — Honeypot placement

D1. Objective

Choose ≤ k hosts (never the entry point or the target) so that likely attacker routes are intercepted early and cheaply.

D2. The implemented greedy score (`ai/optimizer.py`)

score(n) = (path_freq(n) + 1/(hops(n, target) + 1) + degree(n)) ÷ deployment_cost(n)

Term	Meaning
path_freq	How many of the 4 algorithms' paths contain n (0–4)
1/(hops+1)	Proximity to the asset (unweighted hop distance)
degree	Connectivity, a centrality proxy
deployment_cost	Simulated cost: Web 2, App 3, Employee 1, Admin 2, File 2

Score every candidate, sort in descending order, and take the top k. That is greedy selection.

D3. Real output (A*, k = 2)

Candidate	Freq	Prox	Deg	Cost	Score
Web Server	4	0.333	3	2	**3.667 ✓**
Employee PC	0	0.333	3	1	**3.333 ✓**
Application Server	4	0.5	3	3	2.5
File Server	0	0.5	3	2	1.75
Admin PC	0	0.5	2	2	1.25

Worked check for Web Server: (4 + 1/3 + 3) / 2 = 7.333 / 2 = 3.667.

D4. Strengths and weaknesses

- **Strengths:** transparent, fast, budget-aware, and tied directly to the predicted routes.
- **Weaknesses:**
- Nodes are scored independently, so redundancy between decoys is ignored.
- Degree and low cost can outweigh path frequency. Employee PC is selected even though no predicted path uses it.
- The weights are design choices, not tuned on data.

D5. Placement-level fitness (Hill Climbing and GA)

F = 4·coverage + 2·proximity + 1·criticality + 1.5·deception − 1·normalised cost

- **Coverage** is the share of plausible routes intercepted.
- Every term is normalised to [0, 1] before weighting.

Greedy scores and fitness values measure **different objectives**. Never compare 3.667 with 6.2667.

D6. Validation

- **Interception check:** `evaluate_interception(path, honeypots)` returns the first honeypot on the path and its index.
- **Route coverage:** how many of the 3 cheapest routes each placement intersects. All three methods score 3/3 here.
- **Experiment Lab (A*, budget 1, 20 runs, seed 0):**

Strategy	Interception rate
None	0%
Random	50%
Greedy	100%
Hill Climbing	100%
Genetic Algorithm	100%

At budget 2, random rises to 85%.

D7. The chokepoint caveat (must be stated)

`Internet` has a single neighbour, `Web Server`. Every route therefore passes through Web Server, and any method that picks it intercepts 100% of the time. The results show the pipeline works. They do **not** show that the method generalises.

Part E — AI and reasoning

E1. AI vs machine learning

- **AI** is the broad field of systems that act rationally: search, planning, reasoning, learning.
- **Machine learning** is one subfield, in which models learn parameters from data.

This project uses classical AI and no ML. There is no trained model.

E2. Search vs prediction

- "Attacker path prediction" here means *model-based* prediction: assuming a rational, cost-minimising attacker, these are its preferred routes.
- It is not statistical prediction from attack data.
- There are no accuracy, precision or recall numbers, and none must be claimed.

E3. Heuristics

- A heuristic is a rule of thumb that guides a search toward good solutions.
- **Admissible** heuristics never overestimate the true remaining cost. The A* heuristic is admissible.
- The greedy deception score is a heuristic *decision rule*. It is not proven optimal.

E4. Agents (Russell & Norvig)

- **Attacker:** a goal-based agent (`AttackerAgent`). It plans a route, then walks it one hop at a time in stepped sessions.
- **Defender:** a utility-based agent (`DefenderAgent`). It maximises a score under a budget.

E5. Bayesian reasoning (`ai/bayesian.py`)

Bayes' rule: $P(H|E) = P(E|H)P(H) / [P(E|H)P(H) + P(E|\neg H)(1-P(H))]$

- **H:** "this node is compromised on the way to the target".
- **Prior:** min(0.95, 0.2 + 0.5·degree/max_degree). For example, Web Server's prior is 0.2 + 0.5·(3/3) = 0.7.

Evidence (each is either observed or not):

Evidence	P(E|H)	P(E|¬H)
On the predicted path	0.9	0.25
Fortified with a honeypot	0.15	0.6
Is the critical asset	0.7	0.4

If the evidence is *not* observed, the complements are used instead (1 − each value).

Assumption: the evidence items are conditionally independent, which lets the updates be chained.

Worked example (Web Server):

1. Start at the prior: 0.7.
2. On the path (observed): 0.63 / 0.705 = 0.894.
3. Not fortified (use 0.85 vs 0.4): 0.947.
4. Not the critical asset (use 0.3 vs 0.6): **0.899**.

Other results:

- Database: prior 0.7, posterior 0.969.
- Route risk: $1 - \Pi(1 - p_i) = 0.9999$.

Interpretation: these figures are illustrative. **The likelihoods are assumptions, not measurements.**

E6. Adversarial reasoning

Minimax and Alpha-Beta search a sequential, zero-sum game with a hand-designed utility. Describe this as *adversarial search*. It is not a computed Stackelberg or Nash equilibrium.

E7. Model evaluation

There is no learned model, so there is no train/test split or accuracy to report. Evaluation consists of:

- **correctness tests:** optimality against brute force and NetworkX, and Alpha-Beta matching Minimax;
- **simulated comparisons:** the Experiment Lab.

E8. Avoiding unsupported claims

Never say:

- "the model was trained";
- "it detects real attacks";
- "it is 100% accurate";
- "it uses deep learning";
- "it is a real-time threat monitor".

Part F — Complete implementation walkthrough

F1. Frontend (`frontend/src`)

Stack: React 18, TypeScript, Vite 5, Tailwind 3, React Flow 11.

App shell (`App.tsx`):

- sidebar navigation;
- a header with a "Simulated environment" badge and the API/DB status (it polls `/api/health` every 30 s);
- shared state: the analysis settings and this session's simulation history.

Pages:

Page	File	Purpose
Dashboard	`pages/DashboardPage.tsx`	Real metrics, latest result, recommendations
Network Topology	`pages/NetworkTopologyPage.tsx`	Inspector and host table
Attack Path Analysis	`components/SimulationPanel.tsx`	Main simulation workspace
Honeypot Placement	`pages/HoneypotPlacementPage.tsx`	Greedy vs Hill Climbing vs GA
Advanced Analysis	`AdvancedAIPanel.tsx`	Game search and Bayesian risk
Defense Planning	`PlanningPanel.tsx`	Forward and backward plans
Experiment Lab	`ExperimentLab.tsx`	Repeated runs and strategy comparison
Saved Scenarios	`ScenariosPage.tsx`	Persisted topologies and experiments
Methodology	`AboutPage.tsx`	Techniques, metrics, limitations

Graph (`components/NetworkGraph.tsx`):

- custom nodes with a role glyph and a text tag (Entry, Target, Candidate, Decoy, Host);
- columns laid out by hop distance, ordered by barycentre;

- the analysed route drawn in gold.

API client (`api/client.ts`): typed `apiGet` / `apiPost` . Error messages from the backend's `detail` field are shown to the user.

F2. Backend (`backend/`)

Layer	Location	Responsibility
Entry point	`app/main.py`	FastAPI app, CORS, lifespan (creates tables; a database failure is non-fatal)
Routes	`app/api/routes/`	`health` , `network` , `simulation` , `session` , `ai` , `planning` , `experiments` , `scenarios`
Schemas	`app/schemas/`	Pydantic models validate every request
Services	`app/services/`	e.g. `simulation_service.run_simulation` ; no HTTP code
Algorithms	`ai/` , `simulation/`	Pure Python plus NetworkX
Persistence	`app/models/scenario.py` , `app/db/session.py`	SQLAlchemy 2; SQLite by default, PostgreSQL via `DATABASE_URL`

F3. API flow of one simulation

1. The user clicks **Run Simulation**, which calls `SimulationPanel.handleRun` .
2. `runSimulation()` sends `POST /api/simulate` with JSON `{algorithm, start_node, target_node, honeypot_count}` .
3. The route `simulate()` validates the request with `SimulationRequest` . An unknown algorithm or a budget above 5 returns 422.
4. `run_simulation()` builds the graph with `build_network()` , or `graph_from_scenario()` when a `scenario_id` is given.
5. The attacker side runs: `run_all_algorithms()` and `select_attacker_path()` .
6. The defender side runs: `plan_defense()` calls `score_candidates()` .
7. The outcome is computed: `evaluate_interception()` , `alternative_paths()` and `generate_explanations()` .
8. The service returns a `SimulationResponse` as JSON.
9. The UI renders it in `ResultPanel` and `NetworkGraph` , and the run is added to the history list and the dashboard.

F4. Error handling

Status	Cause
404	Unknown node or scenario
409	Duplicate scenario name
422	Validation error
503	Database unavailable

The UI shows a "Simulation failed" panel containing the server's message. The header pill shows "API unreachable · retry" when the backend is down.

F5. Database

Tables: `scenarios`, `scenario_nodes`, `scenario_edges`, `experiments`.

Topology validation runs on save:

- node IDs are unique;
- edge endpoints exist;
- weights are positive;
- start and target exist and differ.

Tables are created with `create_all`. No migration tool is used.

F6. Docker

`docker-compose.yml` runs three services:

- **db:** postgres:16-alpine, with a `pg_isready` healthcheck and a named volume;
- **backend:** port 8000, starts after the database is healthy, has its own health check;
- **frontend:** Vite, port 5173.

Secrets come from `.env`, which is git-ignored.

F7. How to run and test

```
docker compose up -d --build                # needs POSTGRES_PASSWORD in .env
cd backend && .venv/Scripts/python -m uvicorn app.main:app --port 8000
cd frontend && npm run dev                  # http://localhost:5173
cd backend && .venv/Scripts/python -m pytest -q           # 170 passed
cd frontend && npx tsc -b --noEmit && npx vitest run && npm run build   # 44 passed
```

Part G — Evaluation and cybersecurity

G1. Testing methodology

Unit tests cover the algorithms on hand-built graphs with known answers, including brute-force and NetworkX comparisons.

API tests use FastAPI's `TestClient`. Persistence tests run on a temporary SQLite database.

Frontend: Vitest with Testing Library, with the API mocked.

Manual checks on the live application:

- each page in the browser;
- curl against every endpoint;
- the Docker stack.

G2. Metrics actually computed

- path cost, hops, nodes explored;
- interception rate, attack success rate, average steps before interception;
- fitness, honeypot count;
- per-run execution time in ms. This is machine-dependent and not used as a result.

G3. Baselines

- **None** (no deception);
- **Random** placement (seeded).

These are the only baselines. No comparison with commercial tools was made.

G4. Reproducibility

- The network is fixed and the attacker is deterministic.
- The random strategy, Hill Climbing and the GA are all seeded.
- The same request gives the same response.

G5. Threat-model limitations

- a single 7-host topology;
- hand-set weights;
- an attacker who is rational, fully informed and cannot detect decoys;
- undirected links;
- no vulnerability data;
- interception counts as success, but real detection also depends on monitoring.

G6. False positives

Real honeypots can be touched by scanners, misconfigured scripts or curious staff. Mitigations: allow-listing, correlation with other telemetry, and tuning. None of this is modelled here.

G7. Decoy isolation

In a real deployment, a honeypot must be isolated (separate VLAN, egress filtering) so that an attacker cannot use it as a pivot. This is out of scope, because nothing is deployed.

G8. API security

In place:

- Pydantic validation at the boundary;
- CORS restricted to configured origins, with the `*` wildcard rejected;
- no secrets in the frontend;
- no command execution or unsafe deserialisation.

Not in place: authentication and rate limiting. The tool is a local simulation; a live integration would require both.

G9. Safe simulation boundaries

- no real IPs or hosts;
- no scanning;
- no exploits;
- no credential handling;
- no traffic capture.

All data is synthetic and clearly labelled in the UI.

Part H — Viva questions and answers (120)

B = basic, **I** = intermediate, **A** = advanced.

Concepts

1. **(B) What problem does your project solve?** It decides where a limited number of honeypots should go in a network. It does this by predicting the route an attacker would take to a critical asset, using graph search on a simulated network.
2. **(B) What is cyber deception?** Planting false assets to mislead attackers, so they can be detected, delayed or studied.
3. **(B) What is a honeypot?** A decoy system with no legitimate use. Any interaction with it is a strong signal of malicious activity.
4. **(B) Honeypot vs honeynet?** A honeypot is one decoy. A honeynet is a network of decoys that imitates a real environment.
5. **(B) Why would an organisation use honeypots?** They give high-signal early warning of intruders inside the network, and they can slow attackers down or reveal their techniques.
6. **(B) Low- vs high-interaction honeypots?** Low-interaction honeypots emulate services and are safer but less convincing. High-interaction honeypots are real systems: more realistic, but riskier.
7. **(B) What is an attack path?** An ordered sequence of hosts from the attacker's entry point to the target, where each step crosses a link.
8. **(B) What is lateral movement?** Moving from one compromised host to others inside a network, towards a goal.
9. **(B) Why use graph theory for network security?** Networks are naturally graphs: hosts are vertices and links are edges. Questions like reachability, shortest routes and chokepoints become standard graph problems.
10. **(B) Does your system protect a real network?** No. It is a simulation and decision-support tool on a synthetic network.

Graph theory

11. **(B) Define G = (V, E).** V is the set of vertices (hosts) and E is the set of edges (links). Here $|V| = 7$ and $|E| = 9$.
12. **(B) Is your graph directed?** No, it is undirected: a link can be crossed both ways. Directed graphs are future work.
13. **(B) What do edge weights mean?** The simulated difficulty of crossing a link. They are hand-chosen assumptions.

14. **(B) What is the cost of a path?** The sum of its edge weights. For example, Internet → Web → App → Database = 1 + 2 + 5 = 8.
15. **(I) Adjacency list or matrix — which, and why?** An adjacency list (NetworkX). It uses O(V + E) space and gives fast neighbour iteration, which is exactly what search needs.
16. **(B) What is the degree of Web Server?** 3: Internet, Application Server and Employee PC.
17. **(I) Does the graph contain cycles?** Yes, e.g. Web → App → File → Employee → Web. That is why every search tracks visited nodes.
18. **(I) What is a chokepoint here?** Web Server. Internet's only neighbour is Web Server, so every attack path passes through it.
19. **(I) What is reachability, and how is it tested?** Whether a path exists between two nodes. The tests use disconnected graphs and expect an empty path.
20. **(A) How would you model firewall rules?** With directed edges (`nx.DiGraph`), or by removing edges that are blocked under the policy. The search code already only uses `neighbors()` .

Search algorithms

21. **(B) Which algorithms are implemented?** BFS, DFS, UCS and A*, all hand-written in `backend/ai/` .
22. **(B) How does BFS work?** It explores level by level with a FIFO queue and returns the path with the fewest hops.
23. **(B) How does DFS work?** It follows one branch as deep as possible with a stack, then backtracks.
24. **(B) How does UCS work?** It always expands the frontier entry with the lowest cumulative cost g(n), using a min-priority queue.
25. **(B) How does A* work?** It expands the entry with the lowest f(n) = g(n) + h(n), where h estimates the remaining cost.
26. **(I) Which algorithms are optimal?** UCS, for positive weights, and A*, with an admissible heuristic. BFS is optimal only in hop count. DFS is not optimal.
27. **(I) Is UCS informed or uninformed?** Uninformed. It uses no estimate of the distance to the goal.
28. **(I) Difference between UCS and Dijkstra?** UCS is Dijkstra's algorithm run for a single target, stopping as soon as the target is popped.
29. **(I) What heuristic does your A* use?** BFS hop distance to the goal × the minimum edge weight in the graph.
30. **(I) Why is that heuristic admissible?** Every remaining hop costs at least the minimum weight, so the estimate can never exceed the true remaining cost.
31. **(A) Is it also consistent?** Yes. For an edge (u, v), h(u) ≤ w_min + h(v) ≤ w(u, v) + h(v), because hop distances differ by at most 1 across an edge.
32. **(I) Why did A* explore as many nodes as UCS?** The heuristic is weak. With a minimum weight of 1 it is just the hop count, far below the real costs of 2–5, so it guides very little on a 7-node graph.
33. **(B) Nodes explored by each algorithm on your network?** BFS 5, DFS 4, UCS 7, A* 7.
34. **(B) Path each one found?** All four found Internet → Web Server → Application Server → Database, cost 8.
35. **(I) Then why have four algorithms?** To compare strategies and to show the conceptual difference. On a different graph (`TWO_ROUTES` in the tests), BFS picks a 2-hop path costing 10 while UCS and A* pick a 3-hop path costing 3.
36. **(I) Time complexity of BFS and DFS?** O(V + E).
37. **(I) Time complexity of UCS and A* with a binary heap?** O((V + E) log V).
38. **(I) Space complexity?** O(V) for BFS, UCS and A*. DFS also stores a copy of the path per stack entry, so its memory can be higher.

39. **(I) Why does UCS check `if current in visited: continue`?** The heap can contain stale, more expensive entries for a node. Skipping already-finalised nodes is "lazy deletion".
40. **(I) What is the counter in the heap tuple for?** It breaks ties deterministically and stops Python from comparing node names or lists when costs are equal.
41. **(I) Why does UCS stop only when the goal is popped, not pushed?** A cheaper route might still be discovered. Only at pop time is the cost guaranteed minimal.
42. **(B) What happens if the target is unreachable?** All four return an empty path with cost 0, and the API result is `no_path`.
43. **(I) What happens with an unknown node name?** The service raises `UnknownNodeError` and the API returns 404. The search functions themselves return `([], 0, 0)`.
44. **(I) What if start equals goal?** The search returns `([start], 0, 1)`. The UI disables Run in this case.
45. **(A) How did you prove UCS is optimal in tests?** `test_ucs_respects_edge_costs` enumerates every simple path and compares UCS against the true minimum. Another test checks UCS and A* against NetworkX on five random graphs.
46. **(I) What are alternative routes, and how are they computed?** The 3 cheapest loop-free routes, from Yen's algorithm via `nx.shortest_simple_paths`. Their costs are 8, 9 and 10.
47. **(A) Why not just use NetworkX for everything?** For learning and transparency: the hand-written searches expose nodes explored and step order. NetworkX is used where re-implementing adds no value (Yen, hop distances).

Honeypot placement

48. **(B) How are honeypot locations chosen?** Each candidate gets a deception score: (path frequency + proximity + degree) ÷ deployment cost. The top k are selected.
49. **(B) What is path frequency?** The number of the four algorithms' paths that pass through the node (0–4).
50. **(B) What is proximity?** 1 / (hop distance to the target + 1). Nodes closer to the asset score higher.
51. **(I) Why divide by deployment cost?** To prefer value per unit cost, which reflects a limited budget.
52. **(B) Which nodes are selected with budget 2?** Web Server (3.667) and Employee PC (3.333).
53. **(I) Compute the score of Application Server.** (4 + 0.5 + 3) / 3 = 2.5.
54. **(I) Why is Employee PC chosen though no predicted path uses it?** Its degree of 3 and low cost of 1 dominate the additive score. This is a known weakness of the heuristic.
55. **(I) Is the greedy placement optimal?** No. It is a transparent heuristic: candidates are scored independently and redundancy is ignored.
56. **(I) Can a honeypot be placed on the Database?** No. The entry point and the target are excluded from the candidates.
57. **(I) What do Hill Climbing and the GA optimise?** A placement-level fitness: 4·coverage + 2·proximity + 1·criticality + 1.5·deception − 1·normalised cost.
58. **(I) Results of Hill Climbing and the GA?** HC chose {Application Server, Employee PC} and the GA chose {File Server, Web Server}. Both reached fitness 6.2667.
59. **(A) Why do they choose different sets with the same fitness?** The landscape has ties (plateaus), so several placements are equally good. Seeds and search order decide which one is reported.
60. **(A) Can you compare 3.667 with 6.2667?** No. Greedy scores single nodes; fitness scores whole placements, with different terms and weights.
61. **(I) What is local search, and what is its risk?** Repeatedly moving to a better neighbouring solution. It can get stuck in local optima, which restarts mitigate.

62. **(I) Main GA operators?** Tournament selection, crossover that mixes the parents' nodes, mutation that swaps a node, and repair back to exactly k nodes.
63. **(A) How would you make placement exact?** Formulate it as set cover or an integer linear programme: choose ≤ k nodes so that every top-m route contains a decoy. It is NP-hard in general, but tiny instances solve exactly.
64. **(A) What is a Stackelberg security game, and is it implemented?** The defender commits to a (possibly randomised) strategy and the attacker best-responds. It is not implemented; it is future work.

AI and reasoning

65. **(B) Does your project use machine learning?** No. It uses classical AI: search, agents, heuristics, optimisation, adversarial search, Bayesian reasoning and planning.
66. **(B) Then why call it AI?** Those techniques are core AI as defined in standard texts such as Russell & Norvig. AI is not limited to ML.
67. **(I) Is "attacker path prediction" a prediction model?** It is model-based. Assuming a rational, cost-minimising attacker, the search output is its preferred route. It is not learned from data.
68. **(I) What type of agent is the attacker?** Goal-based: it plans a route to the target with search.
69. **(I) What type of agent is the defender?** Utility-based: it maximises a score under a budget.
70. **(I) What is Minimax?** A depth-limited search of a two-player zero-sum game. MAX picks the highest-valued move and MIN the lowest.
71. **(I) What does Alpha-Beta add?** Pruning of branches that cannot change the result, using bounds α and β. The answer stays the same.
72. **(B) Measured Minimax vs Alpha-Beta?** At depth 4: 69 vs 35 states, 14 branches pruned, same move ("move Web Server") and value 2.6667.
73. **(A) Who is MAX and who is MIN?** MAX is the attacker (utility from the attacker's viewpoint). MIN is the defender, who places honeypots or passes.
74. **(A) What makes up the utility?** At terminal states, ±50 for target reached or intercepted. Otherwise, 10/(1+dist) + 2·|compromised| − 4·protection − 0.5·cost.
75. **(I) Is the game a real game-theoretic equilibrium?** No. It is adversarial search with a hand-designed utility, and I describe it that way.
76. **(B) State Bayes' theorem.** P(H|E) = P(E|H)·P(H) / P(E), where P(E) = P(E|H)P(H) + P(E|¬H)(1 − P(H)).
77. **(I) What are H and E in your model?** H: the node is compromised on the way to the target. E: on the predicted path, fortified with a honeypot, is the critical asset.
78. **(I) How is the prior computed?** min(0.95, 0.2 + 0.5·degree/max_degree). Web Server gets 0.7.
79. **(I) What is the Database's posterior?** 0.969, up from a prior of 0.7.
80. **(A) What independence assumption is used?** That the evidence items are conditionally independent given H, so updates can be chained. The route risk also assumes node compromises are independent.
81. **(A) Are these probabilities real?** No. The likelihoods are documented assumptions in `ai/bayesian.py`, and the outputs are illustrative.
82. **(I) What is STRIPS planning?** States are sets of facts. Actions have preconditions, add lists and delete lists. A plan is a sequence of actions that reaches the goal.
83. **(I) Forward vs backward planning?** Forward search moves from the initial state towards the goal. Backward regression moves from the goal towards the initial state.
84. **(B) What plan was found?** place_honeypot(Application Server), place_honeypot(File Server), block_route(Admin PC, Database). That is 3 actions, cost 7.

85. **(A) Is the plan cost-optimal?** Not guaranteed. Both planners are BFS over states, so the plan has the fewest actions within the depth limit.
86. **(I) Why did backward search explore far fewer states (5 vs 48)?** The goal mentions few facts, so regression branches much less than forward expansion.

Architecture and code

87. **(B) What is the architecture?** A React/TypeScript/Vite frontend calls a FastAPI REST backend. The backend's services call pure-Python algorithm modules (NetworkX) and SQLAlchemy persistence. Docker Compose runs the database, backend and frontend.
88. **(I) Why separate routes, schemas and services?** Separation of concerns. Algorithms can be tested without HTTP, and the same service code is reused by simulations, sessions and experiments.
89. **(I) How does the frontend reach the backend in development?** The Vite dev server proxies `/api` to `localhost:8000`. In production, `VITE_API_BASE_URL` sets the API URL.
90. **(I) Which file contains the main simulation logic?** `backend/app/services/simulation_service.py`, function `run_simulation()`.
91. **(I) Where is the network defined?** `backend/simulation/network.py`: `NODE_ROLES`, `EDGES` and `build_network()`.
92. **(I) What does `evaluate_interception` do?** It walks the attacker path in order and returns `(True, node, index)` at the first honeypot, or `(False, None, None)`.
93. **(I) How is request validation done?** With Pydantic models. For example, `honeypot_count` must be between 0 and 5, `algorithm` must be one of the enum values, and `scenario_id` must be ≥ 1. Invalid requests get a 422.
94. **(I) What happens if the database is down?** Startup logs a warning instead of crashing. Simulations still work, the persistence endpoints return 503, and the health endpoint reports `database: unavailable`.
95. **(I) Can you simulate a custom topology?** Yes. Save a scenario with `POST /api/scenarios`, then run `POST /api/simulate` with its `scenario_id`. The UI has a Simulate button on each saved scenario. There is no topology editor in the UI.
96. **(I) How is a scenario topology validated?** Node IDs must be unique, edge endpoints must exist, weights must be positive, and start and target must exist and differ.
97. **(I) Why React Flow?** It is a purpose-built library for interactive node and edge graphs. The project adds custom nodes with role glyphs and tags, so roles never rely on colour alone.
98. **(I) Where does the dashboard get its numbers?** From `GET /api/network`, `GET /api/health`, and the simulations run in the current browser session. Nothing is invented.
99. **(I) Is simulation history persisted?** One-shot runs live in browser memory only. Saved scenarios and experiments are stored in the database.
100. **(I) What does Docker Compose start?** `db` (PostgreSQL 16 with a healthcheck and volume), `backend` (port 8000), and `frontend` (port 5173).

Testing, debugging, security

101. **(B) How many tests are there, and do they pass?** 170 backend tests (pytest) and 44 frontend tests (Vitest). All passed on 2026-10-21, along with the TypeScript check and the production build.
102. **(I) Name an important edge-case test.** `TWO_ROUTES`: BFS must return the 2-hop path costing 10, while UCS and A* must return the 3-hop path costing 3.
103. **(I) How do the frontend tests avoid needing a backend?** They mock `api/client`. React Flow is also mocked, because jsdom cannot measure layout.

104. **(I) Is there an end-to-end browser test suite?** No. The UI flows were verified manually in a browser and against the live API.
105. **(I) What did you fix during recovery?**
    - search crashes on unknown nodes and on edgeless graphs (A*);
    - explanations that hard-coded "Database";
    - no-path runs reported as "target reached";
    - honeypots allowed on the start node;
    - saved scenarios could not be simulated.
106. **(I) How would you debug a wrong path cost?** Reproduce it with a small hand-built graph in a test, print the frontier at each pop, and compare against `nx.shortest_path_length`.
107. **(I) What security measures does the API have?** Input validation, restricted CORS with no wildcard, no secrets in the frontend, no command execution. There is no authentication, because it is a local simulation.
108. **(I) Could this tool be misused offensively?** It only computes paths on a graph you supply, which is standard graph theory. It has no scanning, exploitation or real-network capability.
109. **(I) What are honeypot false positives?** Benign interactions, such as scanners or curious staff, that trigger alerts. They are not modelled here; in practice you would use allow-listing and correlation.
110. **(I) Why must real honeypots be isolated?** So an attacker who compromises one cannot pivot from it into production systems.

Limitations, results and future work

111. **(B) What are your main limitations?**
    - one small topology;
    - hand-set weights and probabilities;
    - an idealised attacker;
    - a greedy heuristic that ignores redundancy;
    - no real data or deployment.
112. **(I) Your results show 100% interception — is that impressive?** No, and I say so. Internet's only neighbour is Web Server, so any method that picks Web Server intercepts every route. The result shows the pipeline works, not that it generalises.
113. **(I) What did the experiment show?** A*, budget 1, 20 runs: no deception 0%, random 50%, greedy, Hill Climbing and GA all 100%. At budget 2, random reaches 85%.
114. **(I) Why run 20 identical runs?** Because the random strategy varies per seed. The deterministic strategies give the same result every run.
115. **(A) What would a fair evaluation look like?** Many generated topologies without chokepoints, multiple attacker models, confidence intervals, and comparison against an exact optimum.
116. **(I) A modelled link is traversable — does that mean it is exploitable?** No. The graph models connectivity and assumed difficulty, not vulnerabilities.
117. **(I) What is your first piece of future work?** A reproducible benchmark of generated topologies, so that placement methods can be compared meaningfully.
118. **(A) Where would ML fit, if ever?** Only with suitable labelled data, e.g. predicting attacker moves from authorised lab telemetry. It would be validated against the current algorithmic baselines.
119. **(A) How would you integrate a live network safely?** Only with explicit authorisation, read-only data sources, an isolated test environment, and authenticated APIs. It must be kept separate from the default simulation.
120. **(B) What is the main contribution of the project?** An explainable, tested pipeline in which the defender's honeypot decision is computed from the attacker's own predicted routes, implemented as a full-stack

application.

Part I — Practical demonstration script

I1. Start (choose one)

- **Docker:** 1. `cd D:\cyber-deception-ai` 2. `docker compose up -d --build` (set `POSTGRES_PASSWORD` in `.env` first). If port 5432 is busy, run it as `POSTGRES_PORT=5434 docker compose up -d --build`.
- **Local:** 1. Backend: `cd backend`, then `.venv/Scripts/python -m uvicorn app.main:app --port 8000`. 2. Frontend: `cd frontend`, then `npm run dev`.

I2. Confirm the services

- Open `http://localhost:8000/api/health`. Expect `"status":"ok"` and `"database":"connected"`.
- Open `http://localhost:5173`. The header pill should read **API v0.1.0 · DB connected**.

I3. Demo flow (about 5 minutes)

1. **Dashboard.** Point out the real metrics (7 hosts, 9 links, 1 critical asset, 5 candidates) and the empty state: "No simulation has run yet."
2. **Network Topology.** Click **Application Server** and show its properties: degree 3, deployment cost 3, and links (Web Server cost 2, Database cost 5, File Server cost 3). Explain the legend.
3. **Attack Path Analysis.** Set A*, Internet → Database, budget 2, and click Run Simulation. Expected result: - the gold route Internet → Web Server → Application Server → Database; - route cost 8 and 7 nodes explored; - Intercepted at Web Server (step 1)**; - the score table: Web Server 3.667, Employee PC 3.333; - alternative routes costing 8, 9 and 10.
4. Switch to **BFS** and run again. The path and cost stay the same (8), but nodes explored drops to **5**. Explain why.
5. **Stepped simulation.** Click **Start Simulation**, then **Step**. The attacker moves to Web Server and is intercepted.
6. **Honeypot Placement.** Click **Evaluate placements**. Show greedy vs Hill Climbing vs GA, fitness 6.2667, route coverage 3/3 for all three, and the assumptions panel.
7. **Advanced Analysis.** Run Alpha-Beta at depth 4. Show 35 states visited and 14 branches pruned, then compare with Minimax at 69.
8. **Experiment Lab.** Set budget 1 and 20 runs, then **Run Experiment**. Show 0%, 50%, 100%, 100%, 100%, then **state the chokepoint caveat**.

I4. Code to show

- `backend/ai/ucs.py` (the core loop);
- `backend/ai/optimizer.py` (`score_candidates`);
- `backend/app/services/simulation_service.py` (`run_simulation`);
- `frontend/src/components/NetworkGraph.tsx`.

I5. Recovering from demo failures

Symptom	Fix
Header shows "API unreachable"	Start the backend, then click retry
"Could not load the network"	Same as above; check `http://localhost:8000/api/health`
Port 5173 or 8000 busy	Stop the other process, or use a different port
Docker database port conflict	`POSTGRES_PORT=5434 docker compose up -d`
UI does not update after code edits under Docker on Windows	`docker compose restart frontend`
Saved Scenarios shows a 503	The database is unavailable; simulations still work

---

Part J — One-page revision sheet

One-line summary: Predict the attacker route on a weighted graph with BFS, DFS, UCS or A*, then place honeypots with a transparent score and optimisers. Everything is simulated and explainable.

Architecture: React + TypeScript + Vite + Tailwind + React Flow → FastAPI + Pydantic → services → `ai/` and `simulation/` (NetworkX) → SQLAlchemy (SQLite or PostgreSQL) · Docker Compose.

Network: 7 hosts, 9 edges. Entry is Internet, target is Database. Web Server is the chokepoint.

Algorithms:

	Structure	Time	Optimal	Explored here
BFS	Queue	O(V+E)	Hops only	5
DFS	Stack	O(V+E)	No	4
UCS	Heap on g	O(E log V)	Yes	7
A*	Heap on g+h	O(E log V)	If h admissible	7

Key numbers:

- Path cost 8; alternatives cost 8, 9 and 10.
- Greedy picks Web Server (3.667) and Employee PC (3.333); interception at step 1.
- Hill Climbing and the GA both reach fitness 6.2667.
- Minimax visits 69 states vs Alpha-Beta's 35 (14 branches pruned).
- Bayesian posterior for Database: 0.969.
- Plan: 3 actions, cost 7.
- Experiment at budget 1: 0 / 50 / 100 / 100 / 100%.
- Tests: 170 backend and 44 frontend, all passing.

Formulas:

- score = (freq + 1/(hops+1) + degree) / cost
- f = g + h

- $P(H|E) = P(E|H)P(H) / P(E)$
- $F = 4 \cdot cov + 2 \cdot prox + 1 \cdot crit + 1.5 \cdot dec - 1 \cdot cost$

Must say:

- no machine learning;
- the probabilities and weights are assumptions;
- 100% interception is due to the chokepoint;
- nothing is deployed and there is no real network.

Most likely questions:

- Why is UCS optimal?
- Why is A* no faster here?
- How does the score work?
- Where is the AI?
- Why 100%?
- What are the limitations?
- What would you do next?